K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

DEPARTMENT : “C.S.E.”

SEMESTER : 2

ASSIGNMENT 1

COURSE NAME:

Data Structures

COURES CODE:

ETCCDS202

FACULTY

MR. DEEPAK KAUSHIK SIR

SUBMITTED BY:

NAME: Himanshu Yadav

ROLL NO: 2501010109

SECTION: “A”

Introduction:

This report covers the implementation and theoretical analysis of fundamental recursive patterns, ranging from simple linear recursion to complex branching and divide-and-conquer strategies.

1. Factorial and Fibonacci Sequences

Recursion is most intuitive when a problem can be defined by its own previous states. Below are the implementations for Factorial and Fibonacci (both naive and optimized).

Implementation (Python)

 # 1. Recursive Factorial

```python
# Recursive Factorial
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
```

 # 2. Naive Recursive Fibonacci

```python
# Naive Recursive Fibonacci
def fibonacci_naive(n):
    if n < 0:
        raise ValueError("Fibonacci is not defined for negative numbers")
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci_naive(n - 1) + fibonacci_naive(n - 2)
```

3. Memoized Recursive Fibonacci (Optimized)

```python
# Memoized Recursive Fibonacci
def fibonacci_memo(n, memo=None):
    if memo is None:
        memo = {}

    if n < 0:
        raise ValueError("Fibonacci is not defined for negative numbers")

    if n in memo:
        return memo[n]

    if n == 0:
        return 0
    if n == 1:
        return 1

    memo[n] = fibonacci_memo(n - 1, memo) + fibonacci_memo(n - 2, memo)
    return memo[n]
```

Complexity Analysis

Algorithm	***Time Complexity***	***Space Complexity***
Factorial	O(n)	O(n)
Fib_Naive	$O(2^n)$	O(n)
Fib_Memo	O(n)	O(n)

2. Tower of Hanoi

The Tower of Hanoi is a classic mathematical puzzle that demonstrates how a complex problem can be broken into three recursive steps: moving n-1 disks, moving the nth disk, and moving the n-1 disks again.

Implementation & Trace (N=3)

```
# Tower of Hanoi
def tower_of_hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} -> {target}")
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, auxiliary, target)

    # Move the nth disk
    print(f"Move disk {n} from {source} -> {target}")

    # Move n-1 disks from auxiliary to target
    tower_of_hanoi(n - 1, auxiliary, target, source)


# Example run
tower_of_hanoi(3, 'A', 'C', 'B')
```

 # Execution:

tower_of_hanoi(3, 'A', 'C', 'B')

Trace for N=3:

* Move disk 1: A -> C

* Move disk 2: A -> B

* Move disk 1: C -> B

* Move disk 3: A -> C (Base move)

* Move disk 1: B -> A

* Move disk 2: B -> C

* Move disk 1: A -> C

Complexity:

The time complexity is **$O(2^n - 1)$**. This is because every increase in n doubles the number of required moves.

3. Recursive Binary Search

Binary search is the quintessential "Divide and Conquer" algorithm, significantly more efficient than linear search for sorted datasets.

Implementation

```
# Recursive Binary Search
def binary_search(arr, left, right, target):
    if left > right:
        return -1  # target not found

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search(arr, left, mid - 1, target)
    else:
        return binary_search(arr, mid + 1, right, target)
```

Recurrence-Based Complexity

The time complexity can be derived using the recurrence relation:

- **Explanation**: In each step, we reduce the search space by half (n/2) and perform a constant amount of work (c) to compare the middle element.
- According to the Master Theorem, this results in a time complexity of O(log n).
- Space Complexity: O(log n) due to the recursive call stack.

THANK YOU!

Himanshu Yadav

2501010109